

Déploiement — FutureKawa

Projet	FutureKawa
Document	Documentation de déploiement
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Vue d'ensemble

FutureKawa est une application **multi-services conteneurisée**. Chaque service dispose de son propre Dockerfile. Le déploiement repose sur Docker Compose en développement et sur Terraform en production (cible : VPS Docker).

Document	Contenu
	Vue globale des services, ports, réseau
	Dockerfiles, registry GitLab, stratégie de tags
	Infrastructure as Code, modules siège et exploitation
	Toutes les variables requises par service

Navigation

#	Document	Contenu
1		Services, ports, flux réseau
2		Images, registry, tags
3		IaC, modules, validation

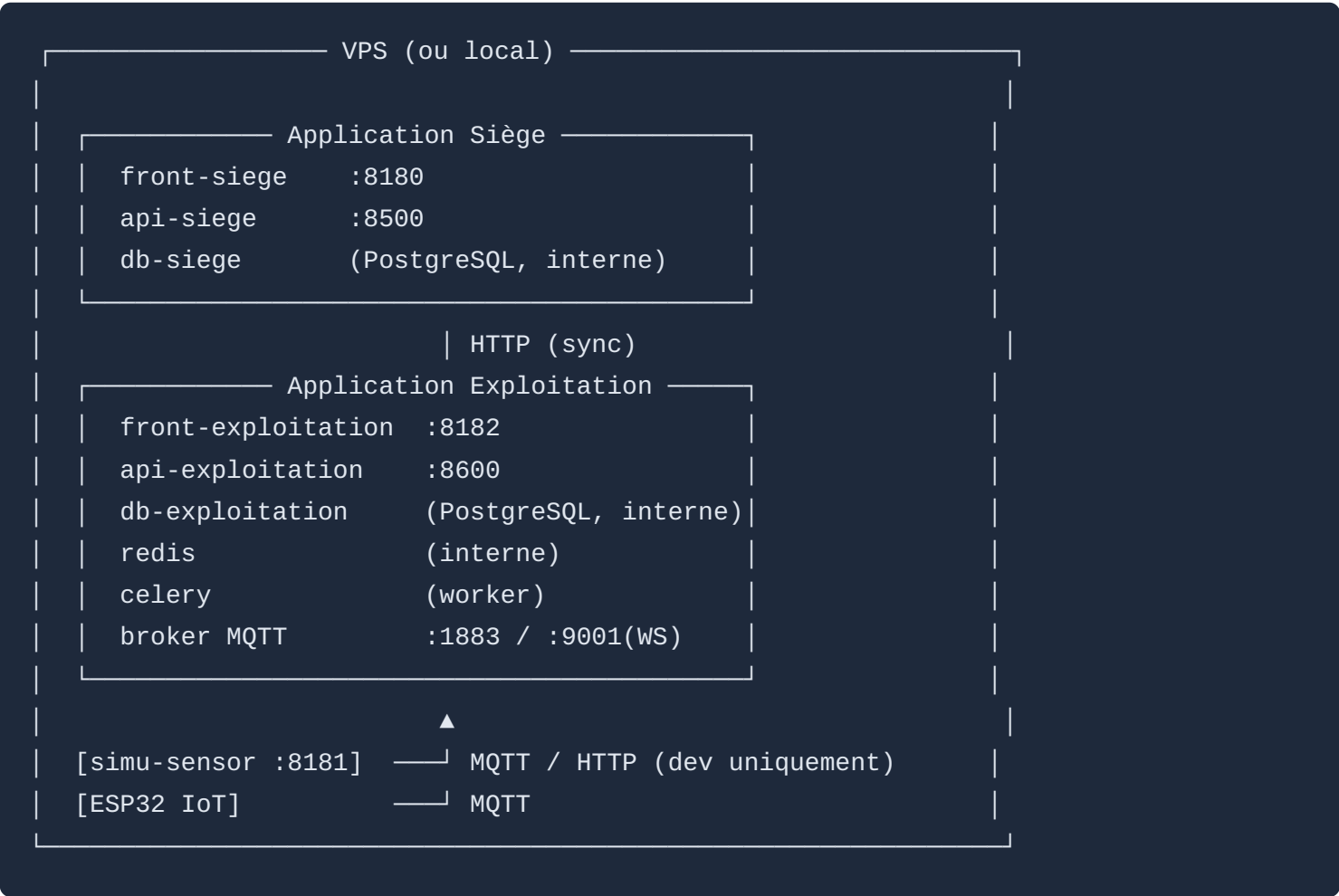
#	Document	Contenu
4		.env par service

01 — Architecture de déploiement

|

Vue globale

FutureKawa est composé de **deux applications indépendantes** déployées séparément, plus un broker MQTT partagé par les plantations.



Services et ports

Application Siège

Service	Port exposé	Description
front-siege	8180	Interface Vue 3 — supervision centrale
api-siege	8500	API Django REST — gestion utilisateurs et sites
db-siege	—	PostgreSQL (réseau interne uniquement)

Application Exploitation

Service	Port exposé	Description
front-exploitation	8182	Interface Vue 3 — gestion plantation
api-exploitation	8600	API Django REST — capteurs, lots, alertes
db-exploitation	—	PostgreSQL (réseau interne uniquement)
redis	—	Cache + broker Celery (réseau interne)
celery	—	Worker Celery (tâches asynchrones)
broker	1883 (MQTT) / 9001 (WS)	Broker MQTT Mosquitto

Services additionnels (dev uniquement)

Service	Port exposé	Description
simu-sensor	8181	Simulateur de capteur Vue 3 (profil demo)

Réseau Docker

Chaque stack possède son propre réseau Docker interne. Les communications inter-stacks (siège → exploitation) passent par les ports exposés sur l'hôte, pas par le réseau Docker.

```
réseau siège :      front-siege ↔ api-siege ↔ db-siege
réseau exploitation: front-exploit ↔ api-exploit ↔ db-exploit
                    ↔ redis ↔ celery ↔ broker
```

Deux instances d'exploitation en démo

Le projet démarre deux plantations distinctes pour la démonstration :

Plantation	Alias	Description
São Paulo	<code>docker - compose.exploit.yml</code>	Exploitation avec simu-sensor (données simulées)
Quito	<code>docker - compose.exploit-1.yml</code>	Exploitation avec capteur IoT ESP32 réel

Le siège supervise les deux via leurs URLs respectives.

|

02 — Docker et images

|

Dockerfiles

Chaque service buildable dispose de son propre `Dockerfile` à la racine de son dossier :

Service	Dockerfile
API Siège	API-siege/Dockerfile
API Exploitation	API-exploitation/Dockerfile
Front Siège	front-siege/Dockerfile
Front Exploitation	front-exploitation/Dockerfile
Broker MQTT	broker/Dockerfile

Le simulateur (`simu-sensor`) utilise aussi un Dockerfile mais ne tourne qu'en développement (profil `demo` du Compose).

Registry GitLab

Les images buildées en CI sont poussées dans le **registry GitLab du projet** :

```
registry.gitlab.com/<groupe>/futurekawa/futurekawa-<service>:<tag>
```

Image	Exemple de tag
futurekawa-api-siege	dev-a1b2c3d / latest
futurekawa-api-exploitation	dev-a1b2c3d / latest
futurekawa-front-siege	dev-a1b2c3d / latest
futurekawa-front-exploitation	dev-a1b2c3d / latest
futurekawa-broker	dev-a1b2c3d / latest

Stratégie de tags

Branche	Tag mobile	Tag SHA
dev	dev-latest	dev-<sha court>
main	latest	main-<sha court>

Le tag SHA est immuable et permet de retrouver exactement quelle version est déployée. Le tag mobile (`latest` , `dev-latest`) est écrasé à chaque build.

Build local

```
# Rebuild toutes les images sans cache
make rebuild

# Build + démarrage avec rebuild
make dev

# Build d'un service spécifique
docker compose -f docker-compose.siege.yml build api-siege
```

Commandes Docker utiles

```
# Voir les images construites
docker images | grep futurekawa

# Logs d'un service
docker compose -f docker-compose.siege.yml logs -f api-siege

# Shell dans un container
docker compose -f docker-compose.exploit.yml exec api-exploitation bash

# Lancer une migration Django manuellement
docker compose -f docker-compose.exploit.yml exec api-exploitation python manage.py
migrate
```

|

03 — Terraform — Infrastructure as Code

|

Présentation

L'infrastructure de production est décrite en **Terraform** dans le dossier `infra/`. Le provider utilisé est `kreuzwerker/docker`, qui pilote Docker sur un VPS distant.

```
infra/
├─ siege/
│   ├── main.tf
│   ├── variables.tf
│   └─ terraform.tfvars.example
└─ exploitation/
    ├── main.tf
    ├── variables.tf
    ├── terraform.tfvars.example
    └─ sites/          ← modules par plantation
```

Modules

`infra/siege/`

Déploie l'application siège sur le VPS : - Container `api-siege` - Container `front-siege` - Container `db-siege` (PostgreSQL) - Réseau Docker interne

`infra/exploitation/`

Déploie une ou plusieurs instances exploitation : - Container `api-exploitation` - Container `front-exploitation` - Container `db-exploitation` - Container `redis` - Container `celery` - Container `broker` - Sous-module par site dans `sites/`

Commandes Terraform

```
# Initialiser (télécharge les providers)
cd infra/siege
terraform init

# Vérifier la syntaxe (utilisé en CI)
terraform validate

# Voir le plan de changements (nécessite un serveur cible)
terraform plan -var-file="terraform.tfvars"
```

```
# Appliquer les changements
terraform apply -var-file="terraform.tfvars"
```

État actuel en CI

Le pipeline CI exécute uniquement `terraform validate` sur les deux modules (stage `deploy`, gate manuel). Aucun `plan` ni `apply` n'est automatique : cela requiert un serveur cible réel et les variables sensibles configurées.

```
# Extrait .gitlab-ci.yml
deploy_prod:
  stage: deploy
  when: manual
  script:
    - cd infra/siege && terraform init -backend=false && terraform validate
    - cd ../exploitation && terraform init -backend=false && terraform validate
```

Configuration

Copier et renseigner les fichiers d'exemple :

```
cp infra/siege/terraform.tfvars.example infra/siege/terraform.tfvars
cp infra/exploitation/terraform.tfvars.example infra/exploitation/terraform.tfvars
```

Les fichiers `terraform.tfvars` ne sont **pas versionnés** (`.gitignore`). Ils contiennent les secrets (mots de passe BDD, clés Django, adresses IP).

|

04 — Variables d'environnement

|

Présentation

Les variables d'environnement sont centralisées dans un fichier `.env` à la racine du projet (non versionné). Le `Makefile` l'inclut automatiquement avec `include .env`.

```
cp .env.example .env
# Éditer .env avec les valeurs réelles
```

Variables globales (toutes stacks)

Variable	Description	Exemple
<code>COMPOSE_PROJECT_NAME</code>	Préfixe des containers Docker	<code>futurekawa</code>

Application Siège — API Django

Variable	Description	Requis
<code>SECRET_KEY</code>	Clé secrète Django (siège)	Oui
<code>DEBUG</code>	Mode debug (<code>True</code> / <code>False</code>)	Oui
<code>ALLOWED_HOSTS</code>	Hôtes autorisés	Oui
<code>DB_NAME</code>	Nom de la base PostgreSQL siège	Oui
<code>DB_USER</code>	Utilisateur PostgreSQL	Oui
<code>DB_PASSWORD</code>	Mot de passe PostgreSQL	Oui
<code>DB_HOST</code>	Hôte PostgreSQL (nom du container)	Oui
<code>DB_PORT</code>	Port PostgreSQL	<code>5432</code>
<code>EMAIL_HOST</code>	Serveur SMTP pour les invitations	Oui
<code>EMAIL_PORT</code>	Port SMTP	<code>587</code>
<code>EMAIL_HOST_USER</code>	Adresse e-mail expéditeur	Oui

Variable	Description	Requis
EMAIL_HOST_PASSWORD	Mot de passe SMTP	Oui
FRONTEND_URL	URL du frontend siège (liens e-mail)	Oui

Application Exploitation — API Django

Variable	Description	Requis
SECRET_KEY_EXPLOITATION	Clé secrète Django (exploitation)	Oui
DEBUG_EXPLOITATION	Mode debug	Oui
ALLOWED_HOSTS_EXPLOITATION	Hôtes autorisés	Oui
DB_NAME_EXPLOITATION	Nom de la base PostgreSQL exploitation	Oui
DB_USER_EXPLOITATION	Utilisateur PostgreSQL	Oui
DB_PASSWORD_EXPLOITATION	Mot de passe PostgreSQL	Oui
DB_HOST_EXPLOITATION	Hôte PostgreSQL	Oui
DB_PORT_EXPLOITATION	Port PostgreSQL	5432
REDIS_URL	URL Redis (Celery broker)	redis://redis:6379/0
EMAIL_HOST	SMTP (invitations exploitation)	Oui
FRONTEND_URL_EXPLOITATION	URL du frontend exploitation	Oui

Broker MQTT

Variable	Description	Requis
MQTT_USER	Nom d'utilisateur MQTT	Oui
MQTT_PASSWORD	Mot de passe MQTT	Oui

Simu-sensor (dev uniquement)

Variable	Description	Défaut
VITE_MQTT_BROKER_URL	URL WebSocket du broker MQTT	ws://localhost:9001
VITE_MQTT_USER	Utilisateur MQTT	admin
VITE_MQTT_PASSWORD	Mot de passe MQTT	—
VUE_APP_API_PAYS_URL	URL de l'API exploitation	—

Synchronisation siège → exploitation

Variable	Description	Requis
EXPLOITATION_API_URL	URL de base de l'API exploitation	Oui
EXPLOITATION_API_TOKEN	Master token pour /api/all-data/	Oui

Le master token est généré via la commande Django :

```
docker compose exec api-exploitation python manage.py create_master_token <nom>
```

|